

CS2013 - Programación III

Ejercicios: TDA, copia y movimiento

Clases, lvalues/rvalues y operadores de asignación en C++

Rubén Rivas Medina

Agosto 2026

Alcance

Duración total estimada: 360 minutos. Los seis ejercicios básicos toman aproximadamente 12 minutos cada uno; los intermedios, 18 minutos; y los avanzados, 30 minutos. Use `new[]` y `delete[]` para el recurso principal; no use contenedores de la biblioteca estándar. Todos los identificadores de C++ se presentan en inglés. Los tiempos suponen que el código indicado como “dado” ya fue entregado y que el estudiante conoce `new[]`, `delete[]` y `std::move`; no son tiempos de primera exposición a esos temas.

Los bloques siguientes usan `assert` (`#include <cassert>`). Los observadores mostrados como **given** pertenecen al código base: se incluyen para poder verificar el resultado sin añadir trabajo al ejercicio.

Nivel básico

Ejercicio 1 (Básico · 12 min): IntStack y copia profunda

El código base ya incluye `push`, `pop` y `top`. Complete constructor, destructor y constructor de copia. Una copia debe tener un arreglo propio.

```
class IntStack {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit IntStack(std::size_t capacity = 8);
    IntStack(const IntStack& other);
    ~IntStack();
    void push(int value);
    void pop();
    int top() const;
    bool empty() const; // given
};
```

Caso de prueba:

```
IntStack source(4);
source.push(10);
source.push(20);
IntStack copy(source);
copy.pop();
copy.push(99);
assert(source.top() == 20);
assert(copy.top() == 99);
```

Ejercicio 2 (Básico · 12 min): GradeBook y asignación por copia

Con constructor, destructor y constructor de copia ya dados, implemente solo el operador de asignación. Debe soportar `a = a`.

```
class GradeBook {
    double* data_;
```

```

    std::size_t size_;
public:
    explicit GradeBook(std::size_t size);
    GradeBook(const GradeBook& other);
    GradeBook& operator=(const GradeBook& other);
    ~GradeBook();
    double& at(std::size_t index);
};

```

Caso de prueba:

```

GradeBook source(2);
source.at(0) = 15.0;
GradeBook copy(1);
copy = source;
copy.at(0) = 20.0;
copy = copy;
assert(source.at(0) == 15.0);
assert(copy.at(0) == 20.0);

```

Ejercicio 3 (Básico · 12 min): IntBuffer con crecimiento

El constructor y destructor están dados. Implemente `resize` y `pushBack`. Cuando se llene el arreglo, duplique su capacidad y copie los enteros.

```

class IntBuffer {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
    void resize(std::size_t newCapacity);
public:
    explicit IntBuffer(std::size_t capacity = 2); // given
    void pushBack(int value);
    int at(std::size_t index) const;
    std::size_t size() const; // given
    ~IntBuffer();
};

```

Caso de prueba:

```

IntBuffer buffer(1);
for (int value = 0; value < 5; ++value) buffer.pushBack(value * 10);
assert(buffer.size() == 5);
assert(buffer.at(0) == 0);
assert(buffer.at(4) == 40);

```

Ejercicio 4 (Básico · 12 min): CircularQueue y orden lógico

Las operaciones de cola ya están dadas. Implemente solo el constructor de copia; preserve el orden de salida aunque `front_` no sea cero.

```

class CircularQueue {
    int* data_;
    std::size_t front_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit CircularQueue(std::size_t capacity);
    CircularQueue(const CircularQueue& other);
    void enqueue(int value);
    int dequeue();
};

```

Caso de prueba:

```
CircularQueue source(3);
source.enqueue(1); source.enqueue(2); source.enqueue(3);
assert(source.dequeue() == 1);
source.enqueue(4);
CircularQueue copy(source);
assert(copy.dequeue() == 2);
assert(copy.dequeue() == 3);
assert(copy.dequeue() == 4);
```

Ejercicio 5 (Básico · 12 min): Name y copia de char*

El constructor, destructor y duplicate están dados. Implemente únicamente el constructor de copia para un nombre almacenado en un único char*.

```
class Name {
    char* text_;
    static char* duplicate(const char* text);
public:
    explicit Name(const char* text); // given
    Name(const Name& other);
    ~Name(); // given
    void setFirst(char value); // given
    const char* cStr() const; // given
};
```

Caso de prueba:

```
Name source("Ana");
Name copy(source);
copy.setFirst('L');
assert(std::strcmp(source.cStr(), "Ana") == 0);
assert(std::strcmp(copy.cStr(), "Lna") == 0);
```

Ejercicio 6 (Básico · 12 min): AttendanceRecord y asignación encadenada

El constructor, destructor y copia están dados. Implemente el operador de asignación para que `third = second = first` funcione.

```
class AttendanceRecord {
    bool* present_;
    std::size_t count_;
public:
    explicit AttendanceRecord(std::size_t count);
    AttendanceRecord(const AttendanceRecord& other);
    AttendanceRecord& operator=(const AttendanceRecord& other);
    ~AttendanceRecord();
    void mark(std::size_t student, bool value);
    bool isPresent(std::size_t student) const; // given
};
```

Caso de prueba:

```
AttendanceRecord first(2), second(2), third(2);
first.mark(0, true);
third = second = first;
second = second;
assert(second.isPresent(0));
assert(third.isPresent(0));
```

Nivel intermedio

Ejercicio 7 (Intermedio · 18 min): Text y movimiento

El constructor, destructor y operaciones de copia están dados. Implemente las dos operaciones de movimiento para transferir el puntero y dejar el origen vacío y válido. Relacione esos cinco miembros con la regla de cinco.

```
class Text {
    char* chars_;
    std::size_t length_;
public:
    Text(); // given
    explicit Text(const char* source); // given
    ~Text(); // given
    Text(const Text& other); // given
    Text& operator=(const Text& other); // given
    Text(Text&& other) noexcept;
    Text& operator=(Text&& other) noexcept;
    const char* cStr() const; // given
    bool empty() const; // given
};
```

Caso de prueba:

```
Text source("UTEC");
Text moved(std::move(source));
assert(std::strcmp(moved.cStr(), "UTEC") == 0);
assert(source.empty());
Text target("old");
target = std::move(moved);
assert(std::strcmp(target.cStr(), "UTEC") == 0);
assert(moved.empty());
```

Ejercicio 8 (Intermedio · 18 min): IntMatrix y swap

El constructor, destructor y copia ya están dados. Implemente swap, el constructor de movimiento y la asignación por movimiento.

```
class IntMatrix {
    int* data_;
    std::size_t rows_;
    std::size_t columns_;
    void swap(IntMatrix& other) noexcept;
public:
    IntMatrix(std::size_t rows, std::size_t columns);
    IntMatrix(IntMatrix&& other) noexcept;
    IntMatrix& operator=(IntMatrix&& other) noexcept;
    std::size_t rows() const; // given
    std::size_t columns() const; // given
};
```

Caso de prueba:

```
IntMatrix source(2, 3);
IntMatrix target(1, 1);
target = std::move(source);
assert(target.rows() == 2 && target.columns() == 3);
assert(source.rows() == 0 && source.columns() == 0);
```

Ejercicio 9 (Intermedio · 18 min): Polynomial y retorno por valor

El manejo del arreglo ya está implementado. Programe derivative, que crea y devuelve un nuevo polinomio sin modificar el objeto original.

```
class Polynomial {
    double* coefficients_;
    std::size_t degree_;
public:
    explicit Polynomial(std::size_t degree);
    Polynomial derivative() const;
    double evaluate(double x) const;
    double& coefficient(std::size_t exponent); // given
};
```

Caso de prueba:

```
Polynomial source(2); // 3x^2 + 2x + 1
source.coefficient(0) = 1;
source.coefficient(1) = 2;
source.coefficient(2) = 3;
Polynomial derived = source.derivative();
assert(source.evaluate(2) == 17);
assert(derived.evaluate(2) == 14); // 6x + 2
```

Ejercicio 10 (Intermedio · 18 min): GrayImage y objeto temporal

El constructor, destructor y copia están dados. Implemente el constructor de movimiento, la asignación por movimiento e inverted.

```
class GrayImage {
    unsigned char* pixels_;
    std::size_t width_;
    std::size_t height_;
public:
    GrayImage(std::size_t width, std::size_t height);
    GrayImage(GrayImage&& other) noexcept;
    GrayImage& operator=(GrayImage&& other) noexcept;
    GrayImage inverted() const;
    unsigned char& at(std::size_t row, std::size_t column);
    unsigned char at(std::size_t row, std::size_t column) const; // given
    bool empty() const; // given
};
```

Caso de prueba:

```
GrayImage source(1, 1);
source.at(0, 0) = 10;
GrayImage result = source.inverted();
assert(source.at(0, 0) == 10);
assert(result.at(0, 0) == 245);
GrayImage target(1, 1);
target = std::move(result);
assert(target.at(0, 0) == 245 && result.empty());
```

Ejercicio 11 (Intermedio · 18 min): ScoreList y movimiento explícito

Con la regla de cinco ya implementada, agregue makeSample, que devuelve una lista por valor. Instrumente las operaciones de movimiento con un mensaje.

```
class ScoreList {
    int* scores_;
    std::size_t size_;
    std::size_t capacity_;
public:
    void add(int score);
```

```
std::size_t size() const; // given  
};
```

Caso de prueba:

```
ScoreList makeSample() {  
    ScoreList result;  
    result.add(10);  
    result.add(20);  
    return result;  
}  
ScoreList scores = makeSample();  
assert(scores.size() == 2);
```

Ejercicio 12 (Intermedio · 18 min): CourseRoster y copia segura

El constructor, destructor y copia están dados. Implemente swap y la asignación por copia con una copia temporal.

```
class CourseRoster {  
    int* ids_;  
    std::size_t size_;  
    void swap(CourseRoster& other) noexcept;  
public:  
    explicit CourseRoster(std::size_t size = 0);  
    CourseRoster& operator=(const CourseRoster& other);  
    std::size_t size() const; // given  
};
```

Caso de prueba:

```
CourseRoster empty;  
CourseRoster copy(3);  
copy = empty;  
assert(copy.size() == 0);  
copy = copy;  
assert(copy.size() == 0);
```

Nivel avanzado

Ejercicio 13 (Avanzado · 30 min): History para lvalue y rvalue

Use Text como elemento y un arreglo dinámico como almacenamiento. La regla de cinco y el crecimiento están dados; defina las dos sobrecargas de add.

```
class History {
    Text* entries_;
    std::size_t size_;
    std::size_t capacity_;
public:
    void add(const Text& entry);
    void add(Text&& entry);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};
```

Caso de prueba:

```
History history;
Text message("start");
history.add(message);
history.add(Text("stop"));
history.add(std::move(message));
assert(history.size() == 3);
assert(std::strcmp(history.at(0).cStr(), "start") == 0);
assert(std::strcmp(history.at(1).cStr(), "stop") == 0);
```

Ejercicio 14 (Avanzado · 30 min): TaskList y separación de resultados

La regla de cinco, add y complete están dadas. Implemente extractCompleted: devuelve las terminadas y las elimina del original, preservando el orden.

```
struct Task {
    Text title;
    bool completed;
};
class TaskList {
    Task* tasks_;
    std::size_t size_;
    std::size_t capacity_;
public:
    TaskList extractCompleted();
    void add(Text&& title); // given
    void complete(std::size_t index); // given
    std::size_t size() const; // given
    const Task& at(std::size_t index) const; // given
};
```

Caso de prueba:

```
TaskList pending;
pending.add(Text("code"));
pending.add(Text("test"));
pending.complete(1);
TaskList done = pending.extractCompleted();
assert(pending.size() == 1 && done.size() == 1);
assert(std::strcmp(done.at(0).title.cStr(), "test") == 0);
```

Ejercicio 15 (Avanzado · 30 min): TextList y realocación con movimiento

La regla de cinco y la sobrecarga para lvalue están dadas. Implemente el crecimiento y add(Text&&). Al crecer, traslade los elementos con std::move.

```

class TextList {
    Text* data_;
    std::size_t size_;
    std::size_t capacity_;
    void resize(std::size_t newCapacity);
public:
    explicit TextList(std::size_t capacity = 2); // given
    void add(const Text& value); // given
    void add(Text&& value);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};

```

Caso de prueba:

```

TextList list(1);
Text first("one");
list.add(first);
list.add(Text("two"));
list.add(Text("three"));
assert(list.size() == 3);
assert(std::strcmp(list.at(2).cStr(), "three") == 0);

```

Ejercicio 16 (Avanzado · 30 min): AdjacencyMatrix y transposición

Use un grafo dirigido pequeño con un arreglo dinámico de bool; el manejo del recurso está dado. Implemente transposed, que devuelve las aristas invertidas.

```

class AdjacencyMatrix {
    bool* edges_;
    std::size_t vertices_;
public:
    explicit AdjacencyMatrix(std::size_t vertices);
    void connect(std::size_t from, std::size_t to);
    AdjacencyMatrix transposed() const;
    bool connected(std::size_t from, std::size_t to) const; // given
};

```

Caso de prueba:

```

AdjacencyMatrix graph(3);
graph.connect(0, 2);
AdjacencyMatrix transposed = graph.transposed();
assert(graph.connected(0, 2));
assert(!graph.connected(2, 0));
assert(transposed.connected(2, 0));

```

Ejercicio 17 (Avanzado · 30 min): FileBuffer como tipo solo movable

Modele un búfer de bytes de propietario único. El constructor y destructor están dados; elimine las operaciones de copia e implemente las dos de movimiento. No realice lectura de archivos.

```

class FileBuffer {
    char* bytes_;
    std::size_t size_;
public:
    explicit FileBuffer(std::size_t size); // given
    FileBuffer(const FileBuffer&) = delete;
    FileBuffer& operator=(const FileBuffer&) = delete;
    FileBuffer(FileBuffer&& other) noexcept;
    FileBuffer& operator=(FileBuffer&& other) noexcept;
    ~FileBuffer(); // given

```



```
std::size_t size() const; // given
};
```

Caso de prueba:

```
FileBuffer source(128);
FileBuffer target(std::move(source));
assert(target.size() == 128);
assert(source.size() == 0);
target = std::move(target); // auto-movimiento defensivo
assert(target.size() == 128);
```

Ejercicio 18 (Avanzado · 30 min): DocumentIndex y fusión por movimiento

La regla de cinco y add están dados. Implemente merge, que toma un rvalue, crece solo si es necesario e incorpora sus entradas. El origen queda vacío y válido.

```
class DocumentIndex {
    Text* words_;
    std::size_t size_;
    std::size_t capacity_;
public:
    void add(Text&& word); // given
    void merge(DocumentIndex&& other);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};
```

Caso de prueba:

```
DocumentIndex first, second;
first.add(Text("cat"));
second.add(Text("dog"));
first.merge(std::move(second));
assert(first.size() == 2);
assert(second.size() == 0);
assert(std::strcmp(first.at(1).cStr(), "dog") == 0);
```

Lista de verificación

- Cada clase libera exactamente el recurso que posee.
- Las copias son profundas y la autoasignación es segura.
- Los movimientos dejan el origen válido y destruible.
- Las pruebas incluyen al menos un lvalue y un rvalue cuando el ejercicio lo requiere.